

SENTINEL AI

MULTI-AGENT CRIME INVESTIGATION SYSTEM

Domain: Generative AI, Computer Vision, & Multi-Agent Orchestration

COURSE NAME

G4 GenAI

TEAM MENTOR

Sir Prujith Radhakrishnan

TEAM MEMBER

Jay Beedkar

PERIOD OF THE PROJECT

60 Days (May 2026 - July 2026)

Declaration

The project titled “Sentinel AI - Multi-Agent Crime Investigation System” has been mentored by Sir Prujith Radhakrishnan, organised by SURE Trust, from May 2026 to July 2026, for the benefit of the educated rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the member of the team mentioned below, has worked on it successfully and enhanced his practical knowledge in the domain.

Team Member:

Jay Beedkar _____ Signature

Mentor:

Sir Prujith Radhakrishnan
Team Mentor — SURE Trust

Prof. Radhakumari Challa
Executive Director & Founder
SURE Trust

Table of Contents

1. Executive Summary
2. Introduction & Background
3. Project Objectives
4. Methodology & Implementation
 - 4.1 Multi-Modal Ingestion Flow
 - 4.2 Video Perception (OpenCV & YOLOv8)
 - 4.3 Stateful Orchestration (LangGraph)
 - 4.4 Database & Persistence (SQLite, NetworkX, & Neo4j)
 - 4.5 Technology Stack Table
5. Social & Industry Relevance
6. Learning & Reflection
7. Conclusion & Future Scope

1. Executive Summary

Sentinel AI is a state-of-the-art, autonomous, multi-modal cognitive crime investigation system designed to assist forensic investigators and security specialists. Modern criminal investigations are heavily bottlenecked by data silos and manual correlation. Investigators frequently spend days cross-referencing hundreds of hours of raw CCTV feeds, audio statements, and conflicting witness testimonies. This manual oversight delays case breakthroughs and increases human error.

Sentinel AI addresses these gaps by integrating real-time computer vision (YOLOv8) with a cooperative network of specialized LLM agents coordinated through LangGraph. The system automatically processes raw video uploads, runs object and pedestrian detection to log physical entities, transcribes audio testimonies, and routes everything into a central LangGraph state manager. Specialized AI sub-agents (Evidence Analyzer, Timeline Builder, Contradiction Detector, Suspect Reasoner) process this unified state in parallel to extract timelines, flag logical contradictions, and rank suspects based on Motive, Means, and Opportunity. The entire system is packaged inside a single-process Streamlit interface and replicates relationships to local SQLite and live Neo4j database clusters.

2. Introduction & Background

Digital forensics and crime analytics have entered a data explosion. Police and private security units are flooded with multi-modal data. However, simple keyword indexing or linear AI prompts fail on complex cases. A single LLM prompt cannot analyze hours of video, compare differing statements, and reason about suspect motives without succumbing to context-limit errors or hallucinations. To resolve this, investigators need an agentic paradigm where specialized systems with discrete profiles collaborate under an intelligent orchestrator.

Sentinel AI establishes this agentic paradigm. It acts as an autonomous 'digital detective department.' The system ingests raw multi-modal media, processes the physical evidence, and executes logical reasoning steps in structured loops. It separates the perception (detecting objects and transcribing audio) from the cognitive deduction (reasoning about suspect motives and contradictions).

Key Architectural Innovation Components:

- **Cooperative Sub-Agent Mesh:** Replaces the 'single prompt' model with four specialized sub-agents working in parallel and merging their findings back to the main state.
- **Unified In-Process Execution:** Eliminates the complex overhead of running separate API servers by mounting the entire FastAPI, LangGraph, and YOLOv8 pipeline in-process inside the Streamlit runtime, serving all functions under a single link.
- **Dual-Persistence Graph Synchronizer:** Combines lightweight, zero-configuration local SQLite + NetworkX graphs for out-of-the-box local runs, with an active Neo4j adapter that automatically serializes graph topologies to a live Neo4j database server in real-time.

3. Project Objectives

- **Multi-Modal Evidence Ingest & In-Process Setup:** Design a single Streamlit dashboard that handles raw uploads of witness statement text files, audio testimonies, and CCTV camera footage (.mp4) without requiring separate backend server hosting.
- **Real-Time Video Perception via YOLOv8:** Integrate a frame-sampled YOLOv8 computer vision model to identify, filter (people, cars, bicycles, bags), and log visual entities with exact confidence bounds and timestamps.
- **Stateful LangGraph Orchestration:** Compile a cyclic LangGraph workflow that manages the investigation state, routes tasks in parallel to the Evidence Analyzer, Timeline Builder, and Contradiction Detector, and executes a final Suspect Reasoner reduction.
- **Dual-Mode Graph Database Persistence:** Maintain persistent SQL tables (SQLite) and local relation topologies (NetworkX), and automatically synchronize nodes and edges to a live Neo4j database in real-time when connection keys are present.
- **One-Click Exporter:** Build an automated markdown dossier compiler that allows investigators to download a comprehensive report containing timelines, contradictions, and ranked suspects with a single click.
- **Air-Gapped Local LLM Configuration:** Integrate Ollama client routing (using ChatOllama) so that the entire reasoning loop can run offline using local models (like llama3) for sensitive, classified investigations.

4. Methodology & Implementation

4.1 Multi-Modal Ingestion Flow

The application begins by accepting files or text statements. When a text witness statement is entered, it is immediately appended to the active session state as text evidence. When a media file (like a CCTV video or an audio recording) is uploaded, it is stored in the local 'uploads/' directory. Streamlit's session state acts as the central scratchpad, maintaining the unified evidence log. The ingest process handles errors gracefully, ensuring that corrupted uploads do not disrupt the running session.

4.2 Video Perception (OpenCV & YOLOv8)

CCTV camera feeds represent dense physical data. To extract meaning from a video file, Sentinel AI implements an active OpenCV capture loop combined with a YOLOv8 Nano model (via the 'ultralytics' package). Running frame-by-frame inference on dense video is CPU-intensive. To resolve this, the perception module samples the video at a rate of 1 frame per second (FPS). For each sampled frame, the YOLOv8 model evaluates bounding boxes and identifies classes. The system filters for relevant classes (such as 'person', 'car', 'truck', 'bicycle', 'backpack', 'handbag', 'suitcase') and logs the detected entity with its timestamp (relative to the video runtime), location ('CCTV Frame'), and prediction confidence. These logs are appended directly to the evidence list, giving the reasoning agents exact physical timeline points.

4.3 Stateful Orchestration (LangGraph)

The reasoning core is built on LangGraph. The orchestrator defines an AgentState TypedDict containing the global evidence log, timeline, list of contradictions, suspect rankings, and current status. When the investigation starts, the orchestrator compiles and runs a stateful graph workflow:

1. Node 'analyze_evidence' (Evidence Analyzer): Processes all text evidence (statements and video logs) to extract key entities.
2. Parallel Nodes 'build_timeline' (Timeline Builder) & 'detect_contradictions' (Contradiction Detector): These nodes execute in parallel. The Timeline Builder sequences all event descriptions chronologically. The Contradiction Detector cross-references all testimonies to highlight logical conflicts (such as differing timestamps or mismatched suspect visual descriptions).
3. Node 'reason_suspects' (Suspect Reasoner): Merges the outputs of the previous nodes, evaluates suspect motives, means, and opportunities, and compiles the final ranked suspect list.

4.4 Database & Persistence (SQLite, NetworkX, & Neo4j)

To ensure persistent case memory, the database manager handles three distinct persistence layers:

1. SQLite Local Tables: On startup, Sentinel AI initializes local SQLite tables ('entities' and 'relations') to store persistent case files and entity mappings on disk.
2. NetworkX directed graph representation: Handles in-memory graph construction. It allows the Streamlit dashboard to fetch node-link data structures and plot color-coded Matplotlib charts directly in dark mode.

3. Neo4j Real-Time Sync: If 'NEO4J_URI' and 'NEO4J_PASSWORD' are configured in the environment, the database manager initiates a live Neo4j driver. On startup, it synchronizes any local SQLite graph data to Neo4j. In real-time, as the investigation agents discover new nodes (e.g. John, Visitor Clara, Golden Falcon) or relationships (e.g. COLLUDED_WITH, IMPLICATED_IN, GUARDED), the db manager pushes Cypher queries to write them directly into the running Neo4j database.

4.5 Technology Stack Table

Component Layer	Selected Technologies & Packages
Multi-Agent Framework	LangGraph Pregel Runtime, LangChain Core Core API
Generative AI Backends	Groq Llama 3.3 70B (Cloud API) & Local Ollama Llama 3 (Offline client)
Computer Vision & Media	YOLOv8 Nano (Ultralytics), OpenCV (cv2), PyTorch, Torchvision
Graph & Database Engines	SQLite3 (Relational), NetworkX (Memory Graph), Neo4j Graph Database client
User Interface Dashboard	Streamlit Dashboard Framework (Matplotlib visualizer, custom Dark Theme)
Environment & Package Deployment	Python-dotenv (credentials loader), Pip, Dockerfile container configuration

5. Social & Industry Relevance

Sentinel AI has significant relevance in both municipal law enforcement and enterprise security operations. Legal departments and policing units are currently bottlenecked by backlogs. Investigators spend a significant percentage of their time performing data entry and watching video tapes. By automating the extraction of timelines and flagging testimonies contradictions, Sentinel AI reduces this administrative load, allowing human detectives to focus their efforts on critical reasoning and case breakthroughs.

Furthermore, the system prioritizes data security and privacy. In criminal investigations, transmitting sensitive witness statements or CCTV videos to external third-party cloud APIs poses risk. By providing a configuration that runs entirely offline using local Ollama LLMs and local SQLite databases, Sentinel AI ensures that sensitive data remains within physical, air-gapped local drives, complying with strict legal confidentiality standards.

6. Learning & Reflection

Student Jay Beedkar completed this project under the mentorship of Sir Prujith Radhakrishnan during the G4 GenAI course. The project provided a comprehensive learning experience in advanced Python engineering and state-of-the-art AI architectures.

Key learnings and takeaways include:

1. Advanced Agentic Design Patterns: Transitioning from single-prompt architectures to cyclic, stateful workflows using LangGraph.
2. Computer Vision Integration: Implementing frame-sampled YOLOv8 object tracking models and processing raw video feeds in real-time.
3. Database Architecture: Structuring hybrid persistence layers (SQLite and NetworkX) and implementing Cypher query synchronization in Neo4j.
4. Professional Software Engineering: Packaging complex multi-modal applications inside single-process Streamlit runtimes, managing environment credentials, and designing robust exception boundaries.

7. Conclusion & Future Scope

Sentinel AI successfully demonstrates the potential of collaborative multi-agent AI systems in digital forensics. The prototype compiles evidence ingestion, video perception tracking, stateful agent timeline reasoning, and interactive graph mapping into a single, cohesive, out-of-the-box local dashboard. The project was completed successfully within the 60-day roadmap.

Future Scope & Extensions:

1. Optical Character Recognition (OCR): Integrate OCR models to read license plates and text on objects directly from CCTV.
2. Real-Time Audio Transcription: Replace mock statements with a live local Whisper transcription client.
3. Inter-Case Relational Queries: Scale the Neo4j database to support multi-case links, allowing investigators to identify serial patterns across different crime scenes.